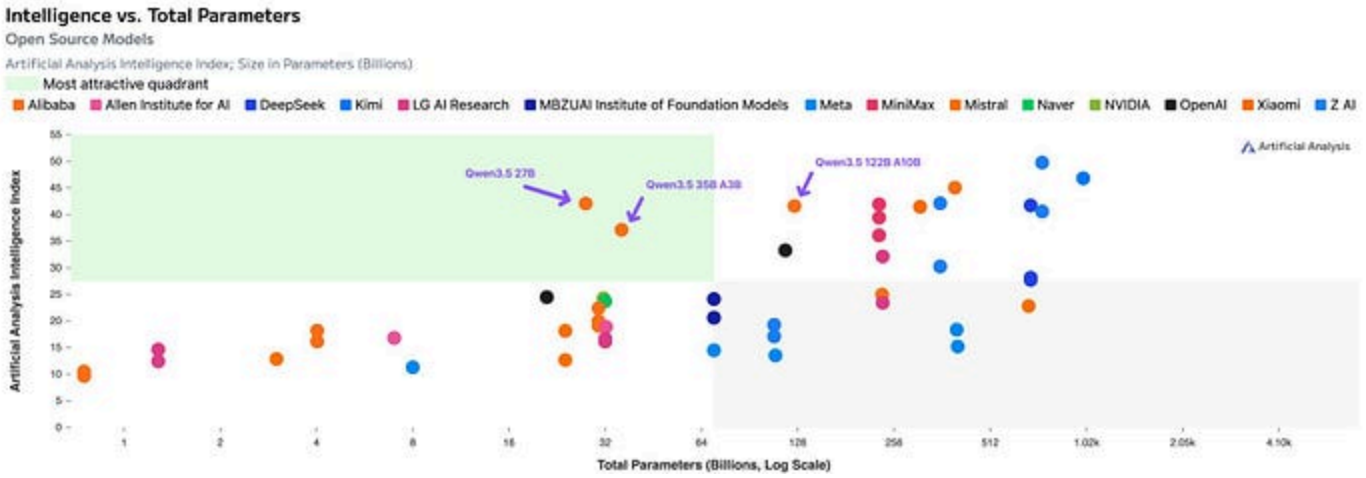
Milestone: 20GB Reached

We've reached 20GB of stored data — thank you for helping us grow!

Patreon Ko-fi Liberapay
Close
Freedium

- ko-fi
- librepay
- patreon

< Go to the original



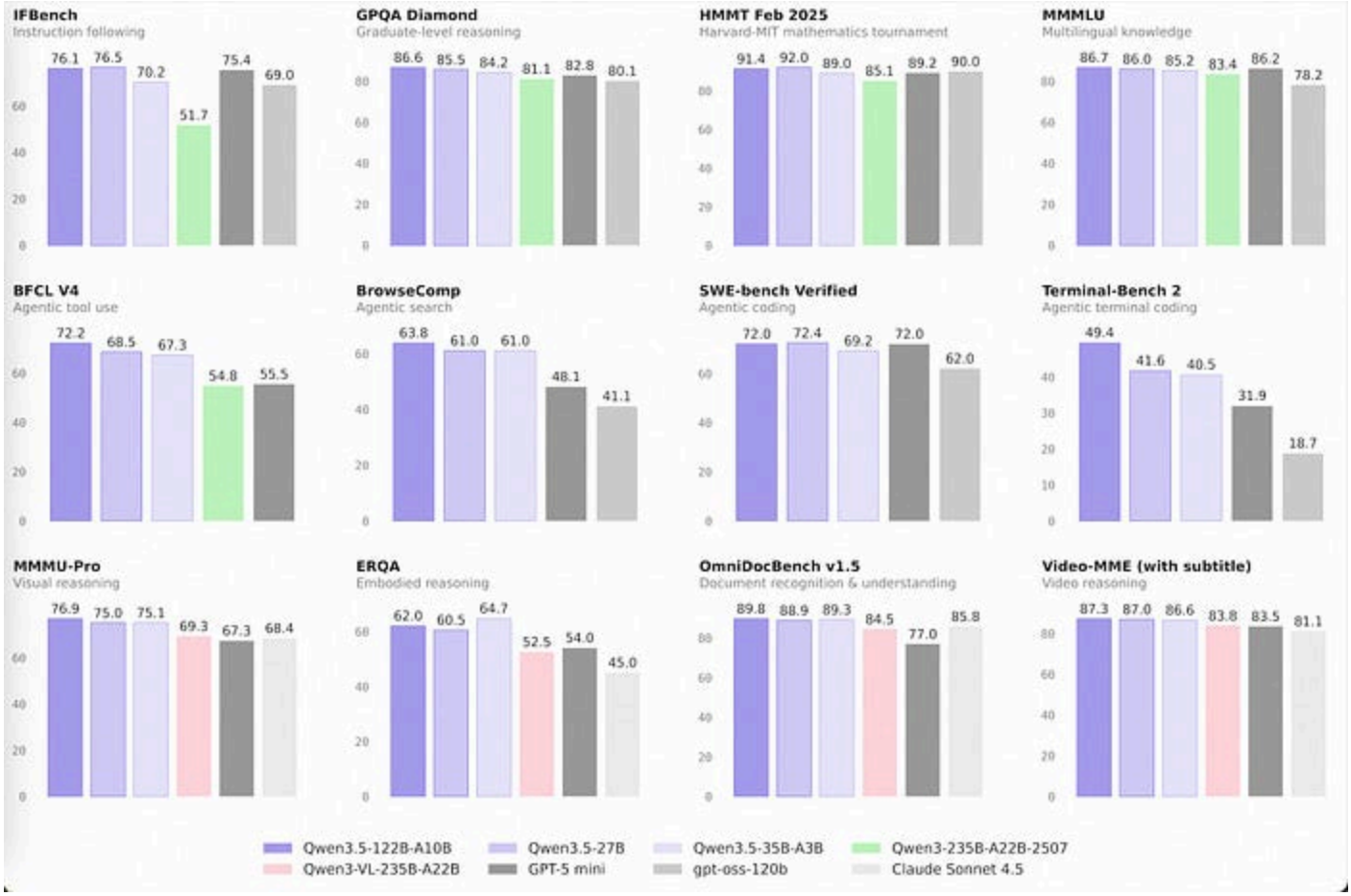
Qwen 3.5 35B-A3B: Why Your $800 GPU Just Became a Frontier Class AI Workstation

I have been running local models for a while now, and I thought I had a pretty good sense of where the ceiling was for consumer hardware.

Agent Native Follow
~10 min read · March 1, 2026 (Updated: March 1, 2026) · Free: No

But when a 35B-parameter model surpasses its 235B-parameter predecessor while activating only 3B parameters per token, it's a turning point for developers waiting on a local models that are viable for production workloads.

It's very impressive to get Sonnet 4.5 level performance out of just 3B activations. That is roughly 8.6% of its total weight.



On a used **RTX 3090** you can grab for around $800, it generates at 112 tokens per second with the full 262K context window, or you can run it on specs like **MacBook Air M4 24GB** with ~15 tokens per second.

Here are other community reported stats for UD-Q4_K_XL (19.7GB), which fits any 24GB card.

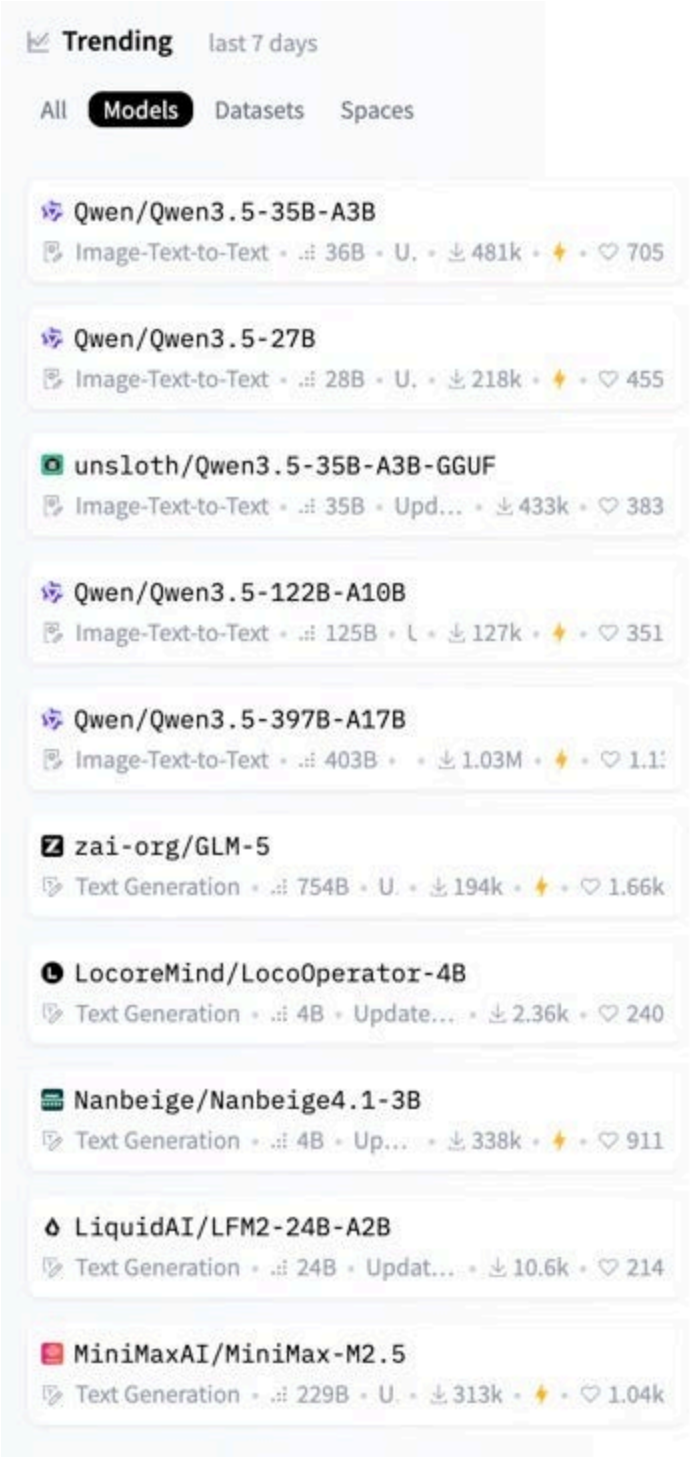
- 2x RTX Pro 6000 Max Q: ~2,600 t/s
- R9700 32GB: 128 t/s Vulkan
- 5090: ~170 t/s
- 4090: 122 t/s
- 3090: ~110 t/s

And lastly, Qwen 3.5 MLX 8bit stats on **M3 Ultra 512GB**:

- Qwen3.5–35B-A3B-8bit: 80.6 t/s (39.3 GB)
- Qwen3.5–122B-A10B-8bit: 42.5 t/s 133.6 GB)
- Qwen3.5–27B-8bit: 21.3 t/s (32.7 GB)

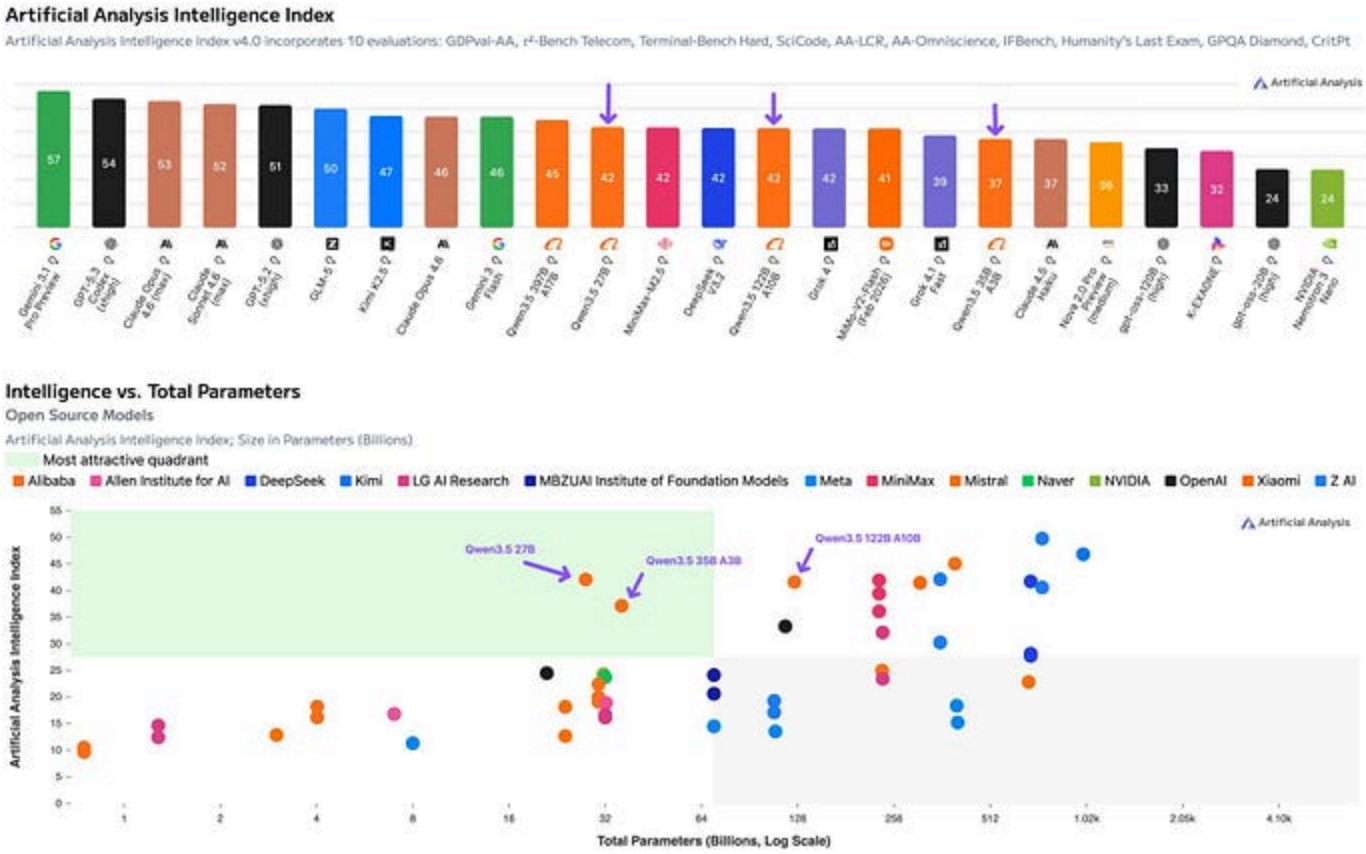
One community member pointed it at Claude Code, gave it a single architecture spec for a complete game, and watched it scaffold 10 files, 3,483 lines of code, debug its own collision detection, and serve a playable game on first load.

This is the reason why all the trending models are Qwen 3.5 on HuggingFace.



I honestly did not expect that from 3B active parameters: autonomous multi-file agentic coding, uncensored automation pipelines, full-context reasoning at interactive speeds.

Price performance ratio is very appealing.



ArtificialAnalysis

The architecture behind this is genuinely new, not just a reshuffled transformer, and most people running it are leaving 40-60% of their throughput on the table because of a few missing flags, the fix is really three CLI arguments away.

Let's break down what Qwen 3.5 35B-A3B actually is under the hood, why the architecture is worth understanding, how to get the performance numbers people are posting, and where the real tradeoffs hide that nobody tweets about.

Why Are We Talking About This Now

Three things happened in the same week, and I think the convergence is what makes this interesting.

First, Alibaba's Qwen team dropped the Qwen 3.5 medium model series with four models under Apache 2.0 that include:

- 35B-A3B
- 27B dense model
- 122B-A10B
- and the flagship 397B-A17B

Qwen 3.5 Medium models outperform all previous Qwen models, beat models that are 6x larger, smarter than Sonnet 4.5 and can run on almost any modern computer completely free.

More interesting part of this announcement was that the model with 3B active parameters now surpasses one with 22B active parameters from the previous generation.

I have seen incremental improvements plenty of times but this definitely changes what **small** means in frontier models.

Second, the Unsloth team shipped optimized GGUF quantizations with day-zero access, including their Dynamic 2.0 quantization that upcasts important layers to 8 or 16-bit while keeping the model compact.

They also caught and fixed a tool-calling chat template bug that was silently breaking function-calling workflows across every quantization uploader.

If you have been scratching your head about why your agent's tool calls kept returning garbage JSON, this was probably it.

Third, the local inference community went absolutely wild with it.

Within 48 hours, there were reports of someone building an entire game autonomously from a single prompt, uncensored agent pipelines signing up for services on autopilot, and throughput optimizations that doubled generation speed with three flags.

What Is Qwen 3.5 35B-A3B

35B is total parameters. A3B means 3 billion active parameters per token., and this is a sparse Mixture-of-Experts model.

What is important to understand here is that you are storing 35B parameters on disk and in VRAM, but at inference time your GPU only crunches the math for 3B.

You get the memory, i.e. the world knowledge, the trained representations, the parametric breadth of a 35B model at the compute cost of something that feels closer to running a 3B model.

Here is how the full lineup stacks up:

Model	Total Params	Active Params	Architecture	Best For	Key Limitation
35B-A3B	35B	3B	MoE + Hybrid Attn	Fast agents, tool-calling, interactive chat	Lower reasoning depth than 27B dense
27B	27B	27B (all)	Dense	Complex coding, deep analysis	15-25 tok/s (4-5x slower)
122B-A10B	122B	10B	MoE + Hybrid Attn	Agentic benchmarks, tool-use (BFCL-V4: 72.2)	Needs 60-70GB+ VRAM
397B-A17B	397B	17B	MoE + Hybrid Attn	Frontier-class (comparable to GPT-5.2)	~214GB at 4-bit quant

All models ship under Apache 2.0.

The 35B-A3B fits in 17–24GB of memory depending on your quantization choice. A Q4_K_M sits around 20GB. That means a single RTX 3090, a 4090, or a 24GB Mac can run this model with the full 262K context window at speeds that feel interactive.

This is great given that we are talking about a model that beats a previous-gen 235B model, running on hardware you probably already own.

Thanks for reading this article. I'm writing a deep-dive ebook on **Agentic SaaS,** the emerging design patterns that are quietly powering the most innovative startups of 2026.

Bookmark it here: [Agentic SaaS Patterns Winning in 2026](), packed with real-world examples, architectures, and workflows you won't find anywhere else.

The Architecture That Makes It Work

Here is the problem everyone working with long-context models has run into: standard transformer attention scales quadratically with sequence length.

Double your context window, quadruple the compute.

We have all felt this pain when trying to do RAG with a 100K+ token context and watching inference slow to a crawl.

The Qwen 3.5 hybrid addresses this by alternating between two attention mechanisms in a **3:1 ratio**:

3 out of every 4 blocks use Gated DeltaNet, a linear attention variant that combines Mamba2's gated decay with a delta rule for updating hidden states.

Each layer compresses the input sequence into a fixed-size state, which gives you near-linear scaling with sequence length. The gating mechanism (a sigmoid that adaptively scales attention outputs) tackles the attention sink problem and prevents the massive activations that blow up training at scale.

1 out of every 4 blocks keeps standard full attention (also gated) to preserve fine-grained token-to-token reasoning.

These layers are what let the model attend precisely to any position in the sequence, and if you have done any code generation work, you know how much that matters for getting imports, variable references, and function signatures right.

This is inline with what was emerging from Kimi Linear's work, though Kimi refines it with channel-wise gating and multi-head latent attention in the full-attention layers.

Seeing two independent teams converge on the same 3:1 ratio tells me something about where the field is heading.

What This Actually Means When You Deploy It

I have spent enough time staring at nvidia-smi during inference runs to appreciate what the 3:1 ratio does in practice.

Three concrete consequences:

(1) Your KV-cache memory drops dramatically. Linear attention layers compress context into fixed-size states instead of storing key-value pairs for every single token. This is the real reason the 35B-A3B runs on consumer GPUs with 262K context, the effective KV-cache footprint is a fraction of what a pure-attention model would need. This is very helpful especially when I am working on agentic pipelines that need to hold entire codebases in context.

(2) Context scaling is nearly flat. Community benchmarks on an RTX 5090 showed the Qwen 3.5 model maintaining throughput with only -0.9% degradation from 512 to 8,192 tokens of context. The older Qwen3 30B-A3B (pure attention + MoE, no DeltaNet) degraded by -21.5% over the same range. If you are building RAG pipelines or long-conversation agents, that is important.

(3) You do pay a throughput tax for the architecture. To be honest, the 3.5 model is roughly 32% slower than the older 30B-A3B in raw generation speed on identical hardware. The DeltaNet layers and larger vocabulary (248K tokens vs. 152K) add overhead but what you are getting in return is context stability. Whether that tradeoff works for you depends entirely on your workload, and I think it is worth being deliberate about which one you pick.

The MoE Layer on Top

On top of the hybrid attention, each feedforward block routes tokens through a sparse MoE layer. The model has 256 expert sub-networks per MoE layer, and a learned router selects the top 9 for each token. That means only about 3.5% of the feedforward parameters are active per token.

The combination of linear attention (reducing memory) and sparse MoE (reducing compute) is what lets the model punch above its weight class.

Local Performance and How to Get It

Default configurations in Ollama, LM Studio, and even naive llama.cpp setups produce 40–70 tok/s on 24GB+ VRAM cards. The optimized setup produces 100–122 tok/s on the exact same hardware.

Because most tools default to f16 KV cache, do not optimize for MoE expert offloading, and skip flash attention flags.

Community benchmarks on an RTX 5080 (16GB) painted a clear picture:

- **Q4_K_M with manual MoE offloading** (–n-cpu-moe 24): ~70 tok/s
- **Q4_K_M with auto-fit**: ~65 tok/s
- **Q8_0 with full offload**: ~35 tok/s

Switching from f16 to q8_0 KV cache gives you a **12–38% throughput boost** while actually using less VRAM, and I can't think of a reason not to use it

The Setup That Actually Gets You 100+ tok/s

If you have 24GB+ VRAM (RTX 3090, 4090, or equivalent), here is the configuration that gets you into the triple-digit range.

I have been running something very close to this and can confirm it works:

Copy

```
# Build llama.cpp from source (required -- prebuilt binaries may miss optimizations)

git clone https://github.com/ggerganov/llama.cpp
cd llama.cpp
cmake -B build -DGGML_CUDA=ON -DCMAKE_BUILD_TYPE=Release
cmake --build build --config Release -j

# Launch with optimized flags
./build/bin/llama-server \
-hf unsloth/Qwen3.5-35B-A3B-GGUF:UD-Q4_K_XL \
--cache-type-k q8_0 \
```

```
--cache-type-v q8_0 \
-np  1  \
-ngl 99 \
-fa  \
--ctx-size 65536 \
--jinja  \
--port 8080
```

What each flag does:

- `--cache-type-k q8_0 --cache-type-v q8_0` 12-38% improvement over the f16 default.
- `-np 1` Sets parallel request slots to 1. Important for single-user local inference because it removes the scheduling overhead.
- `-ngl 99` Offloads all layers to GPU. Only reduce this if you are hitting VRAM limits.
- `-fa` Enables flash attention.

With this setup:

- **RTX 3090 (24GB):** 100 tok/s (up from ~50 tok/s with default flags)
- **RTX 4090 (24GB):** 122 tok/s (up from ~70 tok/s)
- **Full 262K context retained**, zero speed loss

You should be able to reproduce these numbers on a similar hardware and setup.

Why Ollama Is Not the Right Tool for This (Yet)

Ollama does not support MoE expert offloading.

When a MoE model exceeds VRAM capacity, Ollama splits at the layer level, it sends entire transformer blocks to either CPU or GPU. So your GPU just sits there idle, waiting for the CPU to finish processing its layers. That is exactly the wrong strategy for MoE architectures.

The correct approach is expert-only offloading (`-ot "exps=CPU"` or `--n-cpu-moe`), which keeps attention, norms, and shared experts on the GPU while only routing the expert FFN weights over PCIe.

Ollama also defaults to f16 KV cache, often has flash attention turned off, and gives you no control over batch size.

You can find more details in the [commnity discussion here.]

The Claude Code + Local LLM Pattern

This is practically useful for my own work.

Claude Code can be configured to point at a local llama-server endpoint instead of Anthropic's API via a tool called claude-code-router.

You get Claude Code's agentic scaffolding, i.e. file management, error recovery, multi-file editing, all the good stuff with a local model providing the generation.

Copy

```
// Minimal claude-code-router config (~/.claude-code-router/config.json)

{
  "providers": [
    {
      "name": "llamacpp",
```

```
      "api_base_url": "http://localhost:8080/v1/chat/completions",
      "api_key": "not-needed",
      "models": ["Qwen3.5"]
    }
  ]
}
```

If you are doing heavy agentic coding, the API costs with a hosted model would add up fast. With this setup, you just let it run. You will be much more willing to experiment with ambitious prompts when you know there is no per-token charge waiting at the end.

Where It Falls Apart

This model is not a silver bullet though, and I have hit real limitations that are worth knowing about before you commit to building on top of it.

For example, when I throw multi-step logic, nuanced code architecture decisions, or deep mathematical proofs at the 35B-A3B, I can feel it struggling in a way the 27B dense does not.

This makes sense when you think about it, 3B active parameters means roughly 10B-equivalent reasoning depth, not 35B. The knowledge is there, but the depth of processing per token is fundamentally limited by how many parameters are doing the work.

So you can try using thinking mode with temperature 1.0 for complex tasks, and for the hard stuff, you can still reach for the dense 27B or a larger model.

I am curious about your experience. Are you running MoE models locally? Have you tried the KV cache q8_0 flags? What are you building with local agentic setups? Drop a comment, I genuinely want to know what is working and what is not for other people.

Bonus Articles

Local LLMs That Can Replace Claude Code

Small team of engineers can easily burn >$2K/mo on Anthropic's Claude Code (Sonnet/Opus 4.5). As budgets are tight, you...

medium.com

GET SH*T DONE: Meta-prompting and Spec-driven Development for Claude Code and Codex

GSD is a spec-driven development workflow + prompt/agent harness that tries to prevent context rot by externalizing...

medium.com

OpenClaw Memory Systems That Don't Forget: QMD, Mem0, Cognee, Obsidian

If your agent has ever randomly ignored a decision you know you told it… it’s not random.

medium.com

ClawRouter: Anthropic charged me $4,660 — How I cut it 70% with smart LLM routing

Last month I opened my credit-card statement and almost threw up. Anthropic charged me $4,660.87, just for AI APIs.

medium.com

Codex 5.3 vs. Opus 4.6: One-shot Examples and Comparison

Just after 9:45 a.m. Pacific on 5 February 2026, Anthropic unveiled Claude Opus 4.6, and 20 minutes later, OpenAI…

medium.com

Kimi K2.5 + Agent Swarms Beat US AI Labs: Open Source vs OpenAI, Anthropic & Google

If you missed this week’s Kimi K2.5 announcement, you have to read on.

medium.com
#qwen-3#claude-opus#claude-sonnet#ai-agent#coding

Reporting a Problem

Sometimes we have problems displaying some Medium posts.

If you have a problem that some images aren't loading - try using VPN. Probably you have problem with access to Medium CDN (or fucking Cloudflare's bot detection algorithms are blocking you).

Problem Description Describe your problem here...

Submit Cancel